Smazy: Product Concept and Working Prototype

A study platform for students who are smart but lazy

Team Smazy | AWS Hackathon | October 13, 2026

1. About This Document

This document explains Smazy from start to finish: the problem it addresses, who it is for, how it works, what each feature does, how the prototype was built, what is still missing, and what comes next. It is written for a reader who has not seen Smazy before.

It brings together three sources. The first is the original concept brief of October 9, 2026, written before anything was built. The second is the pitch deck prepared for the hackathon. The third is the working prototype, a web application that currently runs on the team's own computer at http://localhost:5173. Where the prototype differs from the original concept, this document describes the prototype and says what changed.

Every feature is marked with its status. **[Built]** means it works in the prototype today. **[Partly built]** means some of it works. **[Planned]** means it is designed but not built. Section 15 gathers these into one table.

2. Summary

Smazy is a web-based study platform built around one idea: a student who is concentrating on a lecture slide should never have to leave it. Switching to another browser tab, a separate chatbot window, or a different file costs attention, even when the switch takes only a moment. Smazy brings the help to the slide.

The student uploads the whole course folder once. Smazy sorts every file into two groups, finds the first lecture, and opens it. From then on, everything the student needs is on the slide: a Study Partner to talk to, links to the rest of the course and to the web, an organized record of past conversations, and tools for writing notes directly on the slide.

The name Smazy blends *smart* and *lazy*. It is for students who want to learn well with the least wasted effort, and who see that as a strength.

3. The Problem

Context switching breaks concentration. When a student is focused on one slide and meets something confusing, the usual response is to open a browser tab or a chatbot and ask "what does this mean?" Research on task switching suggests that even brief switches carry an attention cost. Each one interrupts the student's train of thought.

Study conversations with AI become messy. Studying with a large language model produces long conversations in which explanations, follow-up questions, and side questions pile up. When the student later wants the one explanation that finally made sense, they have to scroll through the whole conversation to find it. That search is slow and destroys the momentum they had built.

Course information is scattered. Lecture slides, the syllabus, homework, project guidelines, and rubrics sit in different places in a university's learning management system. Too much information in too many places overwhelms students. This overload is the main problem Smazy sets out to fix.

4. Target Users and Guiding Principles

Smazy is for university students who are smart but lazy: they want to understand the material and do well, with as little friction as possible. It fits best in courses where the lecture slides, syllabus, guidelines, and rubrics are available early, so the student can upload everything at once.

Four principles guide every design decision:

- **Simplicity.** Few choices and few clicks.
- **Orderliness.** Every file and every conversation has a clear place.
- **Flow.** Nothing should pull the student away from the slide they are studying.
- **Learning momentum.** The student should be able to keep going without stopping to search.

5. Why It Matters: Quality Education (SDG 4)

The United Nations' Sustainable Development Goal 4 is Quality Education. It is usually associated with building schools and providing books. Today's students face a newer obstacle: constant distraction and mental fatigue. Smazy supports SDG 4 in three ways.

- **Fairness for everyone.** Too many tabs and disorganized files tire students before they begin. Smazy organizes everything, so every student starts from the same clear desk.
- **Helping every mind.** Students learn differently. Students with ADHD, for example, often find it hard to stay focused on long, plain pages. Smazy breaks help into small pieces: one slide, one conversation, and short answers by default.
- **Protecting mental health.** Trying to concentrate with many windows open is stressful. Smazy keeps everything in one calm place.

Good education does not mean staying up all night. It means learning well without burning out.

6. How Smazy Works: The User Flow

Step 1. Upload the course folder [Built]

The first screen says "Let's study together with Smazy" and asks the student to drop in the whole course folder, as downloaded from their learning management system. The design uses warm paper tones, navy text, and a single coral accent, so that it reads at once as a calm place to study.

Step 2. Smazy sorts the files [Built]

A sorting screen shows Smazy's progress in four steps: reading the files, sorting them into Learn and Course Info, searching Learn for the first lecture, and opening it. The student can watch each file land in its group. This screen did not exist in the original concept. It was added so that the student can see and trust what Smazy did.

Step 3. Personality questions [Planned]

The original concept places a short set of personality questions here, used to tailor the Study Partner to each student (Section 8.5). The prototype skips this step.

Step 4. The first lecture opens [Built]

The student does not have to choose anything. Smazy reads the lecture order from the file names, which follow conventions such as "Lecture 01", "Week 1", "L1", "Wk01", "Ch01", "Mod1", or "01a intro". When a name is unclear, it also looks at the title on the first slide. If it is still unsure, it opens its best guess and asks "Is this the right first lecture?", listing the alternatives.

Step 5. Study on the slide [Built]

The lecture fills the screen, one slide at a time. The Study Partner sits in the corner. Notes, links, the course files, and the web are each one keystroke away. Sections 8 to 12 describe these in detail.

7. How Materials Are Organized

Files are grouped by what the student does with them, not by file type.

- **Learn.** The lecture slides, in order. This is what the student studies slide by slide. **[Built]**
- **Course Info.** Everything that says what is expected and how the course works: Syllabus, Homework, Guidelines, Rubric, Readings, and Books (references). **[Built]**

The following rules were settled while building the prototype:

- **Readings and textbooks belong to Course Info.** The original concept placed readings under Learn. In practice they support the lectures, so they were moved, and a category called "Books (references)" was added.
- **Practice material is left out.** Past exams, tutorials, and solutions are recognized and set aside. The sorting screen says how many files were left out.
- **Duplicates keep the PDF.** Teachers often upload the same file twice, for example a syllabus as both a PDF and a Word document. Smazy keeps only the PDF.

- **The student can correct mistakes.** Any file can be moved to another category from the file list.

File types. PDF files are shown as slides. PowerPoint and Word files are converted to slides automatically. Excel and CSV files open in a read-only sheet view with tabs for each sheet. **[Built]**

How the sorting is done. In the demonstration, sorting uses rules based on file names, folder names, and page counts. Sorting by an AI model is also built and can be switched on, with the rules as a backup.

8. Feature 1: The Study Partner

Every course has one: the classmate who read the syllabus the day it appeared, knows the rubric by heart, and can tell you what the project requires before you have opened the file. Smazy's Study Partner is that classmate, available to every student in every course.

The Study Partner is deliberately not presented as a chatbot. It is a companion with a face and a name that stays with the student while they study. That is what "study partner" means: someone who is there, not a tool that is summoned.

8.1 A companion on the slide [Built]

A small character sits in the corner of the slide. It floats gently, blinks, follows the pointer with its eyes, gives a small nod when the student moves to a new slide, and waves when the pointer is over it. It speaks rarely. It greets the student once ("Hey, I'm Sam. Stuck on something? Press /"), and on a slide the student has already discussed it says "We talked about this one" and shows how many messages are there. In full screen it moves to the top-right corner and drifts slowly.

8.2 Make it yours [Built]

The student can name the Study Partner and choose how it looks. Clicking its face or name opens a window called "Make it yours" with a live preview. The student chooses a character (Blob, Cat, Bear, Bunny, Ghost, or Chick), one of seven colours, and an extra (glasses, a graduation cap, headphones, a bow, a sprout, or a coffee cup). A "Surprise me" button picks a random combination. The characters are drawn to be cute and friendly without looking childish, because the users are university students. The default name is Sam.

8.3 A conversation that looks like messaging a friend [Built]

Pressing the / key, or clicking the character, opens the conversation beside the slide. It looks like a messaging app, not a chatbot window. The student's messages are on the right. The Study Partner's replies arrive as bubbles on the left, beside its face. The header shows "typing..." while it answers, and the student's last message shows "Delivered" and then "Seen". When the conversation is empty, the Study Partner writes first: "Hey! I'm on slide 2 with you, and I've read the course files. What are we stuck on?"

- **It knows where the student is.** Each question is sent with the text of the open slide, the rest of the open file, and the list of course files, so answers are grounded in the student's own material. **[Built]**
- **Quick or Deep answers.** After typing a question, the student chooses the length of the answer with the Left and Right arrow keys. Quick gives the key point in a few lines. Deep gives a full explanation with an example. Every Quick answer has a "Go deeper" button. **[Built]**
- **Voice.** The student can speak a question instead of typing it, and can have any answer read aloud. **[Built]**
- **A classmate's tone.** The Study Partner is instructed to talk like a friendly classmate: plain words, "I" and "we", and an honest "I think so, but check the slide" when it is unsure. **[Built]**

- **Honest about what it is.** It does not call itself a chatbot, but if the student sincerely asks whether it is a person, it says that it is an AI study partner. **[Built]**

8.4 Encouragement that means something [Built]

The Study Partner notices good study habits and says so in a speech bubble. The aim is constructive praise: each line names what the student did and why it helps, and some end with a small challenge. Examples from the prototype:

- After a highlight: "Good catch. You marked the idea itself, not the whole paragraph."
- After linking two slides and writing down why: "Nice link to Lecture 1, slide 12, and you wrote down why. That's how separate lectures turn into one picture."
- After writing a note: "Putting it in your own words. That sticks far better than reading it twice."
- After downloading notes: "Saved with 4 of your own notes and highlights. That file is worth more than the original now."
- At the last slide of a lecture: "That's the last slide of Lecture 3. Solid session. Can you say its main idea in one sentence?"

It is honest as well as kind. After seven highlights on one slide it suggests keeping only the lines the student would want on exam day. To protect concentration, it waits until the student pauses, and routine praise appears at most about once a minute. These lines are written in advance and chosen by rule. They are not generated by the AI model, so the Study Partner reacts to the action, not to the meaning of what was highlighted.

8.5 A partner matched to each student's personality [Planned]

In the full concept, the Study Partner adapts to each student. A short set of questions after the upload identifies the student's personality type, and the answers set how the Study Partner behaves: how proactive it is, how often it speaks up, and what tone it uses. A more introverted student, for example, would get a partner that takes the lead and offers explanations without waiting to be asked. A more talkative student might prefer one that waits, or one that quizzes and debates.

Proactive help must never break concentration. It should take the form of small hints that can be dismissed, such as a note in the margin of a difficult slide, never pop-ups that cover the content. The student should be able to turn this up or down.

A practical way to build this is to turn the personality answers into a few concrete settings, such as how proactive, how detailed, how encouraging or direct, and whether to quiz the student. Those settings are then given to the AI model, which is more reliable than asking it to act out a personality label.

8.6 The AI model behind it, and its limits

The prototype can use a free model that runs on the student's own computer (Gemma, through a program called Ollama), or Claude through the Claude API or Amazon Bedrock. The demonstration uses the free local model, because the team had no credits for a larger one.

This is the main practical limit on the Study Partner concept in the demonstration. The free model is small. It can follow simple instructions about tone and length, but it is too small to adapt reliably to different personality types, to judge when a hint would help, or to avoid mistakes consistently. The team expects that one of the largest current models, such as Claude Opus 5.5, would act as a much better study partner. That expectation has not yet been tested.

9. Feature 2: Quick Links

The second feature removes the searching. Everything connected to the slide can be reached from the slide.

9.1 Link buttons on the slide [Built]

When a lecture slide mentions the syllabus, the homework, the project, or the rubric, Smazy shows a small button on that slide that opens the matching file. These buttons link only to Course Info files. In the prototype they are found by keyword rules. Having the AI model find the connections is planned.

9.2 Linked slides [Built]

Ideas in one lecture often depend on an earlier one. On a lecture slide, the student presses L and types a lecture number and a slide number. A yellow sticky note appears on the slide, linking it to that slide. Pressing L again cancels.

- Hovering over the note shows a preview of the linked slide. Pressing J jumps there, and pressing J again comes back.
- When a slide has several links they are numbered, and the student picks one by number.
- The student can write a private note on why they made the link. It is shown only after jumping, so it works as a reminder and not as clutter.
- A slide reached through a link cannot start another link, so the student never gets lost in a chain.
- Links are included in the downloaded notes as a small label.

9.3 Keyboard shortcuts, like game controls [Built]

Moving around takes about a second. G followed by a number opens that lecture. K followed by a letter opens a Course Info category, for example K then S for the syllabus. Shift with an arrow key moves to the next document in the same category. The backtick key shows the file list. Appendix A lists every shortcut.

The shortcuts were chosen to be safe. None of them clashes with the operating system or the browser, single keys are ignored while the student is typing, and a guide opens with the ? key.

9.4 A mini web browser [Built]

Sometimes the answer is on the web. Pressing G then B opens a small browser over the slide, with the cursor already in the search box. The student can search, type an address, follow links, and go back and forward. The Study Partner can stay open beside it. Pressing Command and the backtick key closes the browser and returns to the same slide, and G then B reopens the page the student left.

Web links on the slides themselves are clickable. Real hyperlinks and addresses that are simply typed on a slide both open in this browser, as do links in the Study Partner's answers.

The browser has limits. Most websites refuse to be shown inside another page, so Smazy fetches each page through its own server. Reading and searching work well. Sites that need a login, and heavy web applications, do not work in it. A button opens the current page in a normal browser tab for those cases.

10. Feature 3: The Organized Conversation Panel

The third feature solves the messy-conversation problem. It is a way to rewind, not another chat.

- **Conversations pinned to slides.** Every slide has its own conversation. A question asked on slide 12 stays with slide 12. Moving to another slide starts fresh, and coming back brings the earlier conversation back. **[Built]**
- **Follow-ups are recognized.** The AI model decides whether each new question starts a new topic or follows up on an earlier answer. In the conversation, a follow-up is shown as a reply to the earlier message. **[Built]**
- **A history tree for each slide.** Pressing Tab in the conversation shows the slide's questions as a tree diagram. Main questions hang from the slide, and follow-ups branch beneath the question they build on. Colour shows the kind of question: what does this mean, how or why does it work, this did not make sense, give me an example, or checking my own idea. Numbers show the order asked. Clicking a box returns to that point in the conversation. **[Built]**
- **Saving what matters.** The student can star the answer that made things click. Saved questions stand out in the tree with a gold outline and a star. **[Built]**
- **"Still confused" flags.** Questions the student did not understand are tagged and shown in their own colour. Gathering them into a review list is still to come. **[Partly built]**
- **Recall by shortcut.** One key that jumps straight to the saved answers. **[Planned]**

- **Automatic study notes.** A short summary for each slide or lecture. **[Planned]**

This feature was described as "not built" in the first version of the pitch deck. Most of it now works.

11. Feature 4: Notes on the Slide

Students already mark up lecture files in tools such as Preview on a Mac. Smazy brings those tools onto the slide, so that no second application is needed. This feature was not in the original concept. **[Built]**

- **Highlighter, pen, marker, and eraser,** with a choice of colours, undo, and clear.
- **Text boxes** placed beside the slide content. They use Times New Roman at 12 points with 1.5 line spacing by default.
- **The same shortcuts as Preview:** Control, Command and T for a text box, H for the highlighter, and P for the pen.
- **Download.** The lecture can be saved as a PDF that includes every highlight, drawing, text box, and link.
- Notes are saved with each slide and are there when the student returns.

12. Full Screen

Pressing F fills the screen with the slide, in the same way as Preview. The tools stay hidden until the pointer moves to the top edge of the screen. The Study Partner stays visible in the corner. **[Built]**

13. Branding, Naming, and Tone

The name. Smazy is short, easy to remember, and carries both halves of its audience's identity in one word. The tagline is "Smazy. Smart about being lazy."

Why "Study Partner". Inside the platform the helper is called the Study Partner, and the words "AI" and "chatbot" are avoided. The reason is emotional connection. "Chatbot" sounds artificial and distant. "Study Partner" sounds like a friend who studies alongside you. The name also sets an honest expectation: like any classmate, a study partner is helpful and sometimes wrong. A line under the message box says so, and reminds the student to check what matters against the course files.

In presentations. When Smazy is presented to judges, universities, or the press, it is described openly as an AI assistant. The Study Partner name is used inside the platform, where the goal is connection. Whatever it is called, it tells the truth if a student asks whether it is an AI.

14. How the Prototype Is Built

The prototype is a web application in two parts. The part the student sees runs in the browser and is written in React and TypeScript. A small server written in Node.js handles sorting, file conversion, the web browser feature, and the connection to the AI model. Slides are drawn with the pdf.js library, the downloadable notes are produced with pdf-lib, and spreadsheets are read with SheetJS.

The student's course files are read in the browser. Notes, links, conversations, and the Study Partner's name and appearance are stored in the browser on the student's own computer. When the free local model is used, nothing leaves the computer except the pages the student opens in the mini web browser.

The team planned the work spec-first in Kiro. A requirements file states what Smazy must do, a design file describes how, and a tasks file breaks the work into steps that can be built one at a time.

The prototype was tested on five real course folders from the team's own finance courses, which contain PDF, PowerPoint, Word, and Excel files. The source code is kept at https://github.com/Theodre21/smazy.

15. What Is Built and What Is Not

Part of Smazy	Status
Upload, sorting screen, and automatic grouping	Built
Finding and opening the first lecture	Built
PDF, PowerPoint, Word, and Excel files	Built
Study Partner companion and customization	Built
Messaging-style conversation, Quick and Deep answers, voice	Built
Encouragement for good study habits	Built (written lines, chosen by rule)
Personality questions and a personality-matched partner	Planned
Proactive hints on difficult slides	Planned
Link buttons to Course Info files	Built (keyword rules; AI linking planned)
Linked slides and keyboard shortcuts	Built
Mini web browser and clickable web links	Built (reading and searching only)
Conversations pinned to slides, follow-ups, history tree, saving	Built
"Still confused" review list	Partly built
Recall of saved answers by shortcut; automatic study notes	Planned
Notes on the slide and PDF download	Built
Full screen	Built
Accounts, cloud storage, and encryption	Planned

16. Limitations of the Prototype

- **It runs on one computer.** There are no accounts and no cloud storage, and nothing is encrypted. The address http://localhost:5173 works only on the computer where Smazy is running.
- **The AI model is small.** The demonstration uses a free local model. It sometimes files a follow-up question under the wrong branch, and it cannot support the personality-matched partner (Section 8.6).
- **The Study Partner is the same for every student.** Only its name and appearance differ.
- **Sorting and link buttons use rules** in the demonstration, not the AI model.
- **PowerPoint and Word files** need LibreOffice or Microsoft PowerPoint installed on the computer in order to be shown.
- **The mini web browser** cannot sign in to websites, and some sites block it.
- **It has not been tested with students.** It has been tried only on five of the team's own courses.

17. Risks and Planned Answers

- **File names differ between universities.** The first lecture and the lecture order are read from file names. The rules handled all five test courses, and the prototype asks the student when it is unsure. An AI model can read unfamiliar naming styles; this is built as an option and needs testing on more universities.
- **The personality questions must be well made.** The Study Partner's behaviour depends on them. They will be written and tested with a strong AI model and with real students.
- **Proactive help can break concentration.** Hints will be small, dismissible, and adjustable, and will never cover the slide.

- **The Study Partner can be wrong.** Answers use the student's own files, the name signals that it can make mistakes, and a stronger model will make mistakes rarer. A study partner that is wrong too often would lose the student's trust, so accuracy remains a priority.
- **Automatic links can be wrong.** Links show a preview before the student jumps, and any link can be removed.
- **Shortcuts can clash with the browser.** This was solved in the prototype by using key sequences and by pausing single keys while the student types. One shortcut, Command and backtick, should be checked on more computers because macOS also uses it to switch windows.
- **Privacy and ownership.** Course materials belong to universities and instructors, and conversations are personal. The prototype keeps everything on the student's computer. The full product will store data so that only the student can read it, using encryption on the student's side.
- **Courses that release materials week by week.** Smazy fits best when everything is available at the start. Adding files during the semester is a later step.
- **The core idea still needs evidence.** The whole product rests on the belief that staying on the slide improves focus. This will be tested with students.

18. Next Steps

1. Finish the conversation panel: the "still confused" review list, automatic study notes, and recall of saved answers with one key.
2. Build the personality questions and connect the answers to the Study Partner's behaviour.
3. Connect a larger AI model, Claude on Amazon Bedrock, and test whether it acts as a better study partner.
4. Let the AI model write the encouragement and find the links between materials, once a stronger model is connected.
5. Add proactive hints that do not interrupt.

6. Add accounts and private cloud storage, and convert files on the server so that nothing needs to be installed.
7. Test with real students, using a real course folder over a real semester.
8. Check the name Smazy: web address, social media handles, and existing products.

19. Conclusion

Smazy began as a concept on paper: keep the student on the slide and bring the help to them. The prototype shows that most of that concept can be built. A student can drop in a real course folder, have it sorted, start at the first lecture, write notes on the slide, link ideas across lectures, look things up without leaving, and talk to a Study Partner that keeps each conversation where it belongs. What remains is the part that makes the Study Partner personal to each student. That part depends on a stronger AI model and on testing with real students, and it is the team's next goal.

Appendix A. Keyboard Shortcuts

Keys	What they do
Left and Right arrows, or Space	Previous or next slide
G, then a number	Open that lecture (G then 2 opens Lecture 2)
K, then S, H, G, R, D, or B	Open the Syllabus, Homework, Guidelines, Rubric, Readings, or Books
Shift with Left or Right arrow	Previous or next document in the same category
` (backtick)	Show or hide the list of course files
/	Open or close the Study Partner conversation
Enter, then Left or Right arrow, then Enter	Send a question as a Quick or a Deep answer
Tab (in the conversation)	Switch between the conversation and the history tree
L, lecture number, Space, slide number, Enter	Link this slide to another slide
L again	Cancel a link that has been started
J	Jump to the linked slide; press again to come back
B	Go back to the slide you came from
G, then B	Open the mini web browser
Command and ` (backtick)	Close the mini web browser
Control, Command and T	Add a text box
Control, Command and H	Highlight the selected text
Control, Command and P	Turn the pen on or off

Keys	What they do
Command and Z	Undo the last note or highlight
F	Full screen on or off
?	Show the shortcuts guide
Esc	Close, cancel, or return to the normal pointer

On Windows, Control and Alt replace Control and Command, and Control replaces Command.

Appendix B. Running the Prototype

The prototype runs on the user's own computer. It needs Node.js version 18 or newer. For the Study Partner to give real answers it also needs either the free Ollama program with a model installed, or a key for Claude. Without either, the Study Partner gives clearly labelled sample replies and everything else still works.

1. Download the code from https://github.com/Theodre21/smazy.
2. In the downloaded folder, copy the file .env.example to a new file named .env.
3. Run the command "npm run install:all" once to install what is needed.
4. Run the command "npm run dev" to start Smazy.
5. Open http://localhost:5173 in Chrome and drop in a course folder.